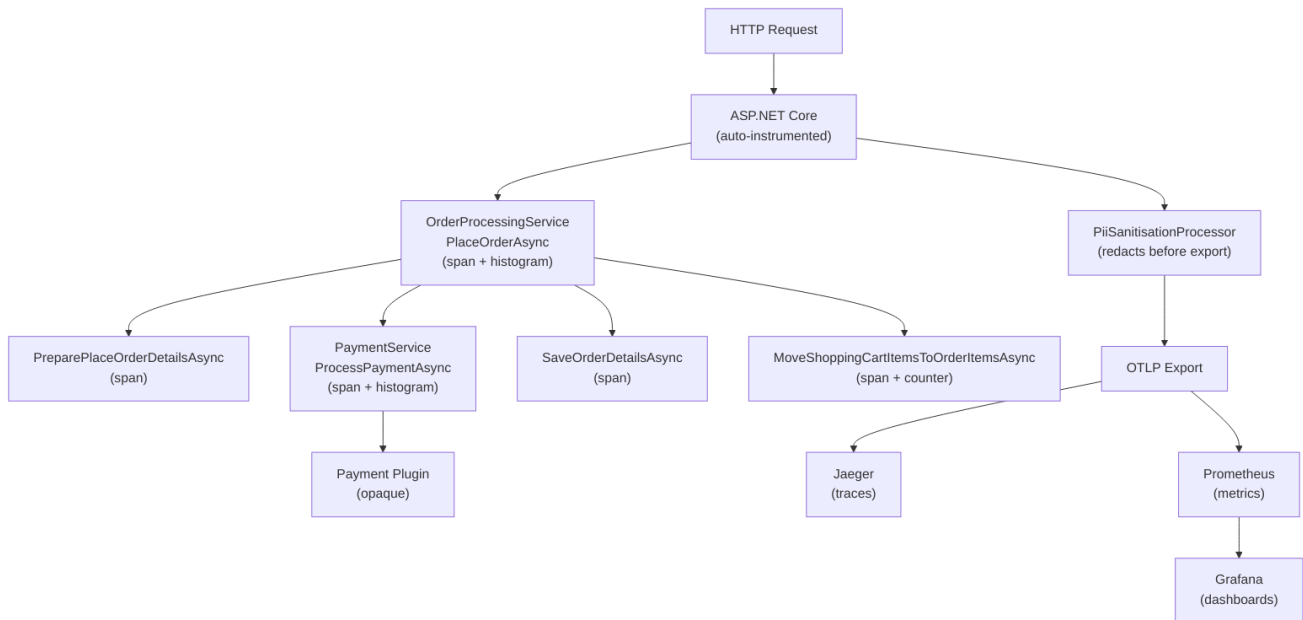


nopCommerce — Observability & Instrumentation

OpenTelemetry-based observability for the nopCommerce order flow, with distributed tracing (Jaeger), metrics (Prometheus), and dashboards (Grafana).

Architecture Diagram



Prerequisites

- **Docker & Docker Compose**
- **k6** (for load testing) — [install guide](#)

Quick Start

1. Start the application + database:

```
docker compose up -d --build
```

2. Start the observability stack:

```
docker compose -f docker-compose.otel.yml up -d
```

3. Complete the nopCommerce installation wizard at <http://localhost> on first run (select MySQL, connection string:

`server=nopcommerce_mysql;port=3306;user=root;password=teste123;database=nopcommerce`).

Viewing the Dashboard

Service	URL	Notes
Grafana	http://localhost:3000	Pre-provisioned dashboard, no login required
Jaeger UI	http://localhost:16686	Search traces by service <code>nopCommerce</code>
Prometheus	http://localhost:9090	Raw metrics queries

The Grafana dashboard **nopCommerce Order Flow** is auto-provisioned and shows order latency histograms, payment duration, error rates, and items-ordered counters.

Running the Load Test

```
k6 run load-test/k6-order-flow.js
```

The test simulates the full order flow (register → add to cart → checkout) with a ramp-up to 10 virtual users. Approximately **15% of orders will fail** by design (`OTEL_SIMULATED_FAILURE_RATE=0.15`) to generate error traces and failure metrics visible in the dashboard.

Project Structure

Path	Description
<code>src/Libraries/Nop.Services/Diagnostics/NopCommerceDiagnostics.cs</code>	Shared <code>ActivitySource</code> and <code>Meter</code> definitions
<code>src/Libraries/Nop.Services/Orders/OrderProcessingService.cs</code>	Instrumented order processing (spans + metrics)
<code>src/Libraries/Nop.Services/Payments/PaymentService.cs</code>	Instrumented payment processing (spans + histogram)
<code>src/Presentation/Nop.Web/Infrastructure/OpenTelemetryStartup.cs</code>	OTel SDK registration via <code>INopStartup</code>
<code>src/Presentation/Nop.Web/Infrastructure/PiiSanitisationProcessor.cs</code>	Redacts PII from trace attributes before export
<code>docker-compose.yml</code>	App + MySQL database
<code>docker-compose.otel.yml</code>	Jaeger + Prometheus + Grafana

Path	Description
observability/prometheus.yml	Prometheus scrape configuration
observability/grafana/provisioning/	Grafana datasource + dashboard provisioning
load-test/k6-order-flow.js	k6 load test script
load-test/dashboard.png	Dashboard screenshot
CRITIQUE.md	Architectural critique (Task 5)